



# Artificial Intelligence

---

## Problem Solving Agent



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## Contents

---

- Problem Solving Agent
- Problem Definition
- Example Problems
- Searching For Solutions
- Problem-solving Performance Measure
- Search Strategies



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## Problem Solving Agent

- The simplest agents were the **reflex agents** - base their actions on a direct mapping from states to actions.
  - cannot operate well in environments for which this mapping would be too large to store
- **Goal-based agents** consider future actions and the desirability of their outcomes
- One kind of goal-based agent called a **problem-solving agent**
- Problem-solving agents use **atomic** representations
  - states of the world are considered as wholes, with no internal structure visible
- Goal-based agents that use **factored** or **structured** representations are usually called **planning agents**



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## Problem Solving Agent

- First Step in Problem Solving: **Goal formulation**
  - based on the current situation and the agent's performance measure
  - Courses of action that don't achieve the goal can be rejected without further consideration
  - Goals help organize behavior by limiting the objectives
  - The agent's task is to find out how to act, now and in the future, so that it reaches a goal state
- **Problem formulation** is the process of deciding what actions and states to consider, given a goal.
- if the environment is **unknown**,

*an agent with several immediate options of unknown value can decide what to do by first examining future actions that eventually lead to states of known value.*



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## Problem Solving Agent

- The process of looking for a sequence of actions that reaches the goal - **search**.
- A search algorithm takes a problem as input and returns a **solution** in the form of an action sequence.
- Once a solution is found, the actions it recommends can be carried out - the **execution** phase.

“*formulate, search, execute*” design for the agent

- Steps:
  - It first formulates a goal and a problem
  - searches for a sequence of actions that would solve the problem, and
  - then executes the actions one at a time.
  - When this is complete, it formulates another goal and starts over.



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## Problem Solving Agent

**function** SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

**inputs:** *percept*, a percept

**static:** *seq*, an action sequence, initially empty

*state*, some description of the current world state

*goal*, a goal, initially null

*problem*, a problem formulation

*state* ← UPDATE-STATE(*state*, *percept*)

**if** *seq* is empty **then do**

*goal* ← FORMULATE-GOAL(*state*)

*problem* ← FORMULATE-PROBLEM(*state*, *goal*)

*seq* ← SEARCH(*problem*)

*action* ← FIRST(*seq*)

*seq* ← REST(*seq*)

**return** *action*



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## Problem Definition (Process of problem formulation)

A problem can be defined formally by five components:

- **Initial state** - the agent starts in
- **Actions**
  - A description of the possible actions available to the agent
  - Given a particular state  $s$ , **ACTIONS(s)** returns the set of actions that can be executed in  $s$ .
- **Transition model**
  - A description of what each action does - **the transition model**
  - Together, the initial state, actions, and transition model implicitly define the **state space** of the problem
  - state space forms a directed **network or graph**
- **Goal test**
  - determines whether a given state is a goal state.
- **Path cost**
  - function that assigns a numeric cost to each path.

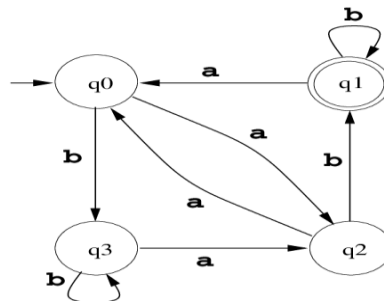


Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## Problem Definition

- A solution to a problem is *an action sequence that leads from the initial state to a goal state*.
- Solution quality is measured by **the path cost function**, and
- an optimal solution has the **lowest path cost among all solutions**.



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



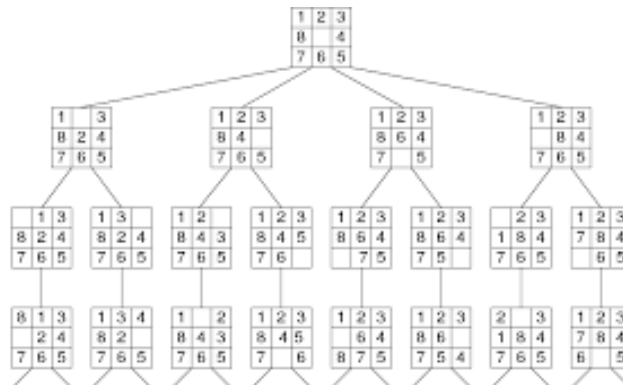
## Example Problems - 8-puzzle

|   |   |   |
|---|---|---|
| 7 | 2 | 4 |
| 5 |   | 6 |
| 8 | 3 | 1 |

Start State

|   |   |   |
|---|---|---|
|   | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

Goal State



9

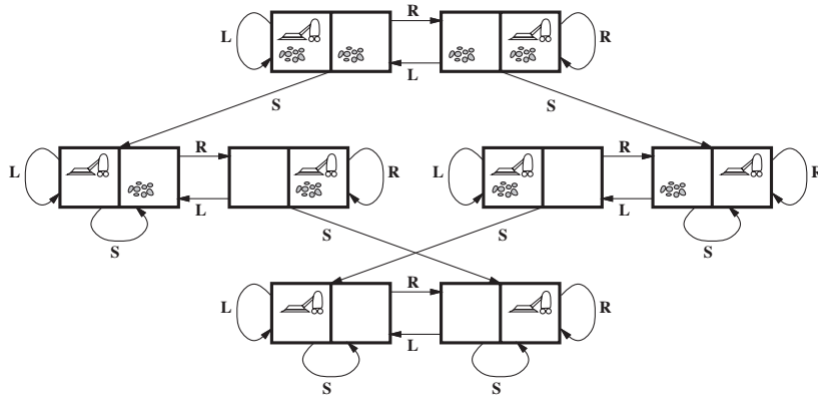


## 8-puzzle

- **States:** the location of each of the eight tiles and the blank in one of the nine squares
- **Initial state:** Any state can be designated as the initial state.
- **Actions:** The simplest formulation defines the actions as movements of the blank space Left, Right, Up, or Down.
- **Transition model:** Given a state and action, this returns the resulting state;
  - for example, if apply Left to the start state, the resulting state has the 5 and the blank switched.
- **Goal test:** checks whether the state matches the goal configuration
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path



## Example Problems - vacuum world

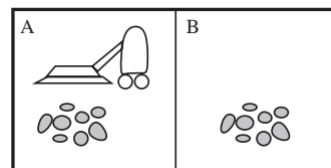


Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam

11

## vacuum world

- **States:** The state is determined by both the agent location and the dirt locations.
- **Initial state:** Any state can be designated as the initial state.
- **Actions:** each state has just three actions: *Left*, *Right*, and *Suck*.
- **Transition model:** The actions have their expected effects, except that moving Left in the leftmost square, moving Right in the rightmost square, and Sucking in a clean square have no effect.
- **Goal test:** This checks whether all the squares are clean.
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.



- 8-queens problem
- route-finding problem

Dr. P. Victor Paul, Indian Institute of



## Searching for Solutions

- *A solution is an action sequence*, so search algorithms work by considering various possible action sequences.
- The possible action sequences starting at the initial state form a search tree with *the initial state at the root*;
  - the *branches are actions* and
  - the *nodes correspond to states* in the state space of the problem.
- Steps in growing Search tree
  - root node of the tree corresponds to the initial state
  - to test whether this is a goal state
  - expanding the current state - generating a new set of states
    - The set of all leaf nodes available for expansion at any given point - the *frontier*
- *This is the essence of search*



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## Tree-search and Graph-search

**function** TREE-SEARCH(*problem*) **returns** a solution, or failure  
 initialize the frontier using the initial state of *problem*  
**loop do**  
   **if** the frontier is empty **then return** failure  
   choose a leaf node and remove it from the frontier  
   **if** the node contains a goal state **then return** the corresponding solution  
   expand the chosen node, adding the resulting nodes to the frontier

---

**function** GRAPH-SEARCH(*problem*) **returns** a solution, or failure  
 initialize the frontier using the initial state of *problem*  
*initialize the explored set to be empty*  
**loop do**  
   **if** the frontier is empty **then return** failure  
   choose a leaf node and remove it from the frontier  
   **if** the node contains a goal state **then return** the corresponding solution  
   *add the node to the explored set*  
   expand the chosen node, adding the resulting nodes to the frontier  
   *only if not in the frontier or explored set*

Three common variants are the

- first-in, first-out or FIFO queue
- the last-in, first-out or LIFO queue
- the priority queue



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam

14



# Tree-search and Graph-search



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam

15



## Performance Measure

*Evaluate an algorithm's performance in four ways*

- **Completeness**: Is the algorithm guaranteed to find a solution when there is one?
- **Optimality**: Does the strategy find the optimal solution (low cost)?
- **Time complexity**: How long does it take to find a solution?
  - No. of nodes generated during the search – **Search cost**
- **Space complexity**: How much memory is needed to perform the search?
  - No. of nodes stored during the search

*Total cost = Search Cost + Path Cost*

*Complexity is expressed in terms of three quantities:*

- b, the **branching factor** or maximum number of successors of any node
- d, the **depth** of the shallowest goal node
- m, the **maximum length** of any path in the state space.



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## Search Strategies

- uninformed search algorithms or **blind search**
  - algorithms that are given **no information about the problem** other than its definition.
  - generate successors and distinguish a goal state from a non-goal state
  - all search strategies are distinguished **by the order in which nodes are expanded**
  - although can solve any solvable problem, not so efficiently.
  - **Breadth-first search, DFS, Uniform-cost search**
- Informed search algorithms or **heuristic search**
  - strategies that know whether **one non-goal state is “more promising” than another**
  - some **guidance on where to look** for solutions
  - **Greedy best-first search, A\* search**



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam



## The End...



Dr. P. Victor Paul, Indian Institute of Information Technology Kottayam

18